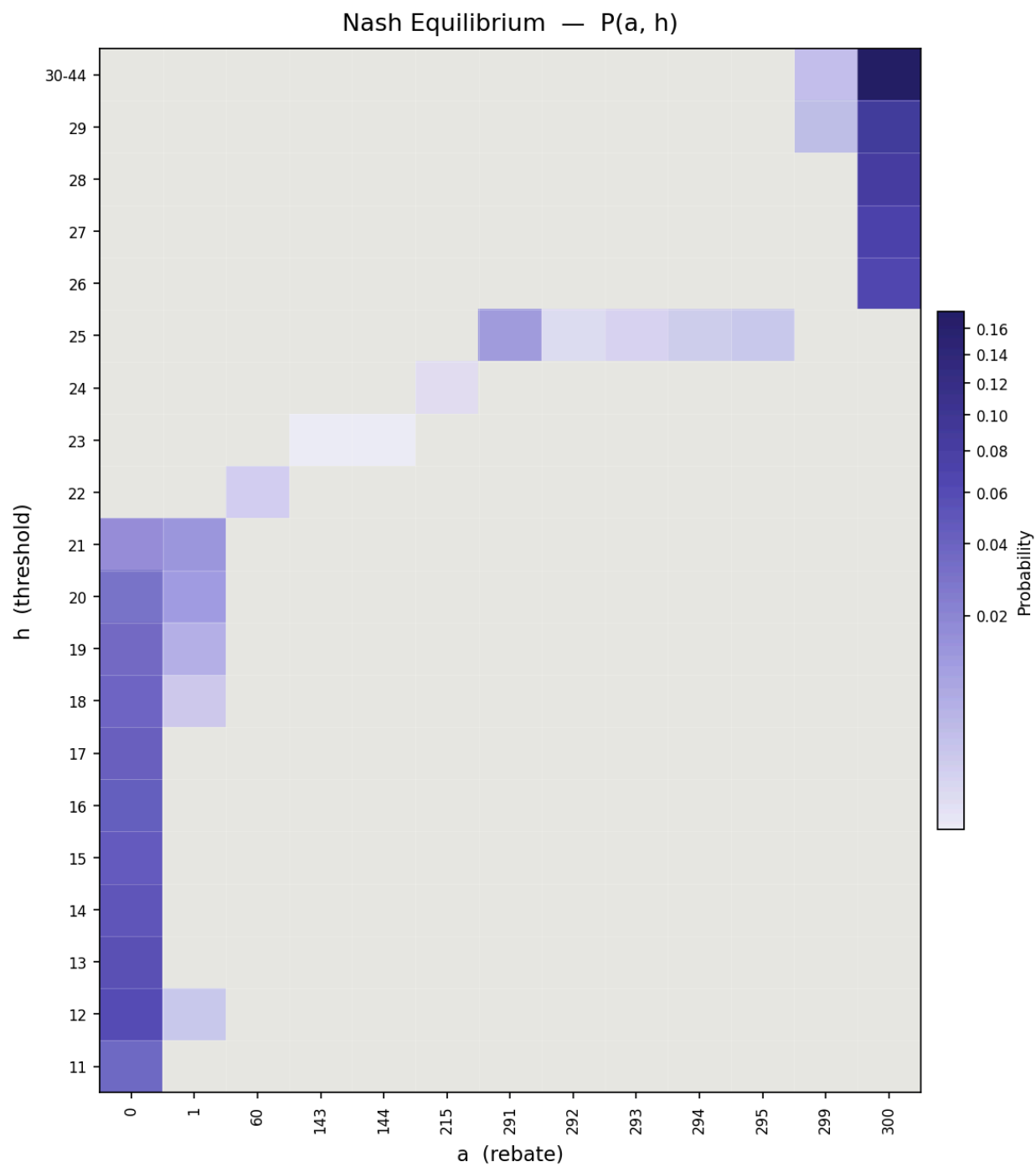
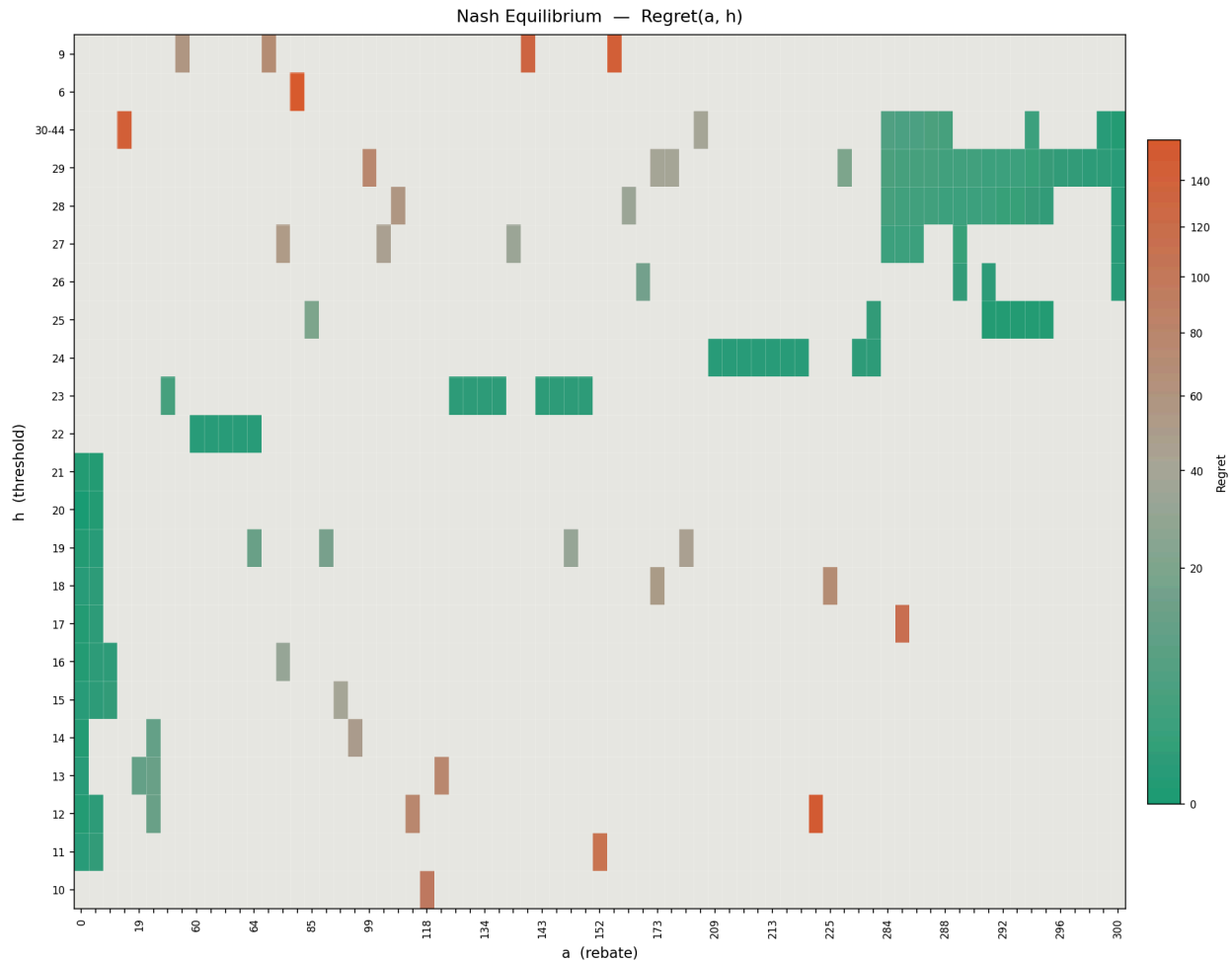


I choose $a=0, h=18$. This was drawn from what I computed to be the unique Nash equilibrium mixed strategy for the game where there is a large, nearly infinite, number of farmers. The case with 8 farmers is addressed in a few pages. C++ and Python codes using parallel programming are pasted afterward with comments. In the equilibrium, (a, h) is ($\sim$ denotes a rough estimate):

(300,30) with probability $\sim 18\%$
(300,29) with probability $\sim 9\%$
(300,28) with probability $\sim 8\%$
(300,27) with probability $\sim 7\%$
(300,26) with probability $\sim 6\%$
 $\sim(293,25)$ with probability $\sim 2\%$
 $\sim(219,24)$ with probability $\sim 0\%$
 $\sim(142,23)$ with probability $\sim 0\%$
 $\sim(61,22)$ with probability $\sim 0\%$
(0,21) with probability $\sim 3\%$
(0,20) with probability $\sim 4\%$
(0,19) with probability $\sim 4\%$
(0,18) with probability $\sim 4\%$
(0,17) with probability $\sim 4\%$
(0,16) with probability $\sim 4\%$
(0,15) with probability $\sim 5\%$
(0,14) with probability $\sim 5\%$
(0,13) with probability $\sim 5\%$
(0,12) with probability $\sim 6\%$
(0,11) with probability $\sim 4\%$





The farmer is expected to have ~418 apples left and low-mid variance on the outcome. I optimised towards maximising expected value in the linear sense, not the probability of having the highest score amongst all farmers on one particular shot. A restatement prompt useful with Gemini, Claude, Arena.AI Agent, ...:

"

The Congested Random Walk

Background

A nearly infinite set of players plays a routing game on a directed $N \times N$ grid. Each player starts at the origin $(0, 0)$ and must reach the destination (N, N) by taking exactly $2N$ steps, moving only right or up at each intersection. Here $N = 50$.

The Strategy

Before the game begins, every player independently chooses a strategy parameterized by a pair of integers, a threshold h , where $h \in [5, 30] \cup [45, 60]$, and a rebate $a \in [0, 300]$. The goal is to find a system-wide mixed strategy—a probability distribution $P(h, a)$ over the valid h, a values—that forms a Nash Equilibrium, minimizing a player's expected total cost.

Cost Generation

The game proceeds in three distinct phases of cost evaluation:

1. Global Base Cost: Each directed edge e in the grid is assigned a global base cost, $C_{\text{base}}(e)$, drawn uniformly at random from the integers $[5, 30]$.
2. Private Tolls: For each player, their personal cost for the edges is modified. The player casts $2a$ independent random $+40$ tolls across the grid. Each toll is assigned uniformly at random to one of the $2N(N+1)$ edges. Let $k_{i,e}$ be the number of tolls assigned to edge e for player i . The private cost for player i on edge e is:
$$C_{\text{private}}^{(i)}(e) = C_{\text{base}}(e) + 40 \cdot k_{i,e}$$
3. Congestion Penalty: Let $f(e) \in [0, 1]$ be the exact fraction of the total player population that traverses edge e during the game. The final evaluated cost for player i taking edge e is:
$$C_{\text{final}}^{(i)}(e) = C_{\text{private}}^{(i)}(e) \cdot (1 + f(e))^3$$

The Routing Policy

Players do not know the congestion fractions $f(e)$ or the global base costs in advance; they walk the grid entirely based on their private costs $C_{\text{private}}^{(i)}$ and their chosen threshold h .

At any node (x, y) , a player looks at the rightward edge e_R and the upward edge e_U :

- * If exactly one edge exists (i.e., at the grid boundaries), the player must take it.
- * If both edges exist, the player evaluates the conditions $C_{\text{private}}^{(i)}(e_R) \leq h$ and $C_{\text{private}}^{(i)}(e_U) \leq h$.
- * If exactly one condition is true, the player takes that corresponding edge.
- * Otherwise (both are true, or neither are true), the player chooses between e_R and e_U uniformly at random.

Objective

Using either parallel C++ numerical or rigorously proven analytic mathematical techniques, find the probability distribution $P(h, a)$ that represents a Nash Equilibrium. The cost to be minimized by choosing h, a is the expected sum of $C_{\text{final}}^{(i)}(e)$ along the walked path, minus a rebate of a .

"

"

Algorithmic Specification: Non-Atomic Routing Nash Equilibrium Solver

1. Architecture Overview

The algorithm solves a non-atomic stochastic routing game on a directed 50×50 grid to find a mixed-strategy Nash Equilibrium distribution over strategy pairs (h, a) , where $h \in [5, 60]$ is a routing threshold and $a \in [0, 300]$ is a fixed rebate. It employs a Multiplicative Weights Update (MWU) engine integrated with a dynamic state-space expansion technique and two-pass Monte Carlo grid simulations.

2. Phase 0: Precomputation of Physics Tables

Before simulating, the algorithm precomputes the exact transition probabilities and conditional expected private costs for every possible strategy against all combinations of adjacent right and upward edge base costs (\$5\$ to \$30\$). It models the \$+40\$ private edge tolls as independent Poisson distributions with $\lambda = 2a/5100$ (as there are 5100 edges), calculating the Probability Mass Function up to $k=40$. These tables resolve the problem's routing logic—taking an edge if exactly one satisfies $\text{cost} \leq h$, or splitting 50/50 otherwise—to yield the exact $P(\text{route right})$ and $P(\text{route up})$. Crucially, the tables also compute the selection-biased conditional expectation $E[\text{private cost} \mid \text{route right}]$ and $E[\text{private cost} \mid \text{route up}]$, factoring in the specific subsets of the Poisson distribution that trigger the routing choice.

3. Phase 1: MWU State-Space Search

To avoid evaluating the computationally massive 301×42 state space simultaneously, the engine maintains an active set of strategies, initialized with a single user-defined seed strategy at 100% probability. At the start of each MWU iteration, it generates a "scout" pool consisting of the active set, local unvisited adjacent neighbors via a 4-way flood-fill on the discrete (h,a) grid, and a fixed number (e.g., 50) of uniformly random global samples from the unvisited domain. The algorithm then runs a batch of Monte Carlo simulations (e.g., 1000 iterations), generating independent topological grids where every edge's base cost is uniformly drawn from integers $[5, 30]$.

4. Monte Carlo Pass 1: Fluid Flow

The grid evaluation is strictly split into two passes, both processing nodes topologically by traversing the grid's diagonals from $d=0$ to $2N$. In Pass 1, the algorithm simulates the infinite-player fluid by pushing the active strategies' probability masses from $(0,0)$ to $(50,50)$. By referencing the precomputed transition probabilities matching the randomized grid's base costs, it divides and routes the incoming probability mass at each node, strictly calculating the expected macroscopic fraction of the population $f(e)$ traversing every edge.

5. Monte Carlo Pass 2: DP Expected Value Evaluation

In Pass 2, with $f(e)$ now frozen, the algorithm evaluates every strategy in the scout pool using a single-pass Dynamic Programming approach. It pushes \$1.0\$ unit of "probe" probability mass for each strategy from the origin to the destination. Applying the Law of Total Expectation, the accumulated expected score pushed to the adjacent node at each intersection is calculated as: $\text{current_score} * P(\text{transition}) + \text{probability_mass_taking_edge} * E[\text{private cost} \mid \text{transition}] * (1 + f(e))^3$. This perfectly isolates an individual atomless player's expected cost against the crowd's congestion penalty without requiring discrete multi-agent simulation.

6. MWU Probability Update

After the Monte Carlo batch completes, each scout strategy's raw Expected Value is finalized by dividing by the destination mass at $(50,50)$ and subtracting its specific rebate a . Any non-active scout strategy whose EV is less than or equal to the minimum EV of the active set is promoted into the active set with an initial probability of 0.003. The algorithm maintains an Exponentially Weighted Moving Average (EMA) of the EV for all active strategies using a $0.9 (\text{new}) + 0.1 (\text{old})$ blend. The probability weight of each active strategy is then updated via the MWU formula: $w_s = P_s \times \exp(-\text{learning_rate} \times (\text{EMA}_s - \min(\text{EMA})))$, where the learning rate decays

continuously via $\$0.25 / \sqrt{1 + \text{iteration}}$. The weights of the active set are then strictly normalized to sum to \$1.0\$.

"

As the number of farmers approaches infinity, the fraction of farmers on each edge in an unmemorised bottoms up dynamic programme has variance approaching 0. Note that h values of 31 through 44 inclusive can be ignored as they are, for all intents and purposes, logically equivalent with just selecting $h = 30$. If instead one wants to strategise for 8 farmers (there were 8 LinkedIn reposts which were required for eligibility) then one could run simulations under those settings and check convergence rates - I saw $a = 0$ strategies dominate. Many of the same numerical insights apply. For 8 farmers I got $a=0$ along with the rough h probabilities:

9	0.0305
10	0.1038
11	0.0882
12	0.0771
13	0.0706
14	0.0636
15	0.0589
16	0.0566
17	0.0536
18	0.0521
19	0.0526
20	0.0531
21	0.0549
22	0.0606
23	0.0627
24	0.0610
25	0.0003

I did not see how to use analytic techniques (which work for the $N = 1, 2, 3$ cases more easily) or store more information about the joint distributions to somehow compute for all possible grids at the same time. Thus, I resorted to using simulations in the following sense. I had a probability distribution over the (a, h) pairs and there were at most a couple hundred active strong ones in the support. Then, on each iteration I randomly generated 10,000 grids. Then ran the dynamic programming to compute the EVs for each pair and generated 50 random (a, h) pairs on top of pairs neighbouring those active. If any had sufficiently low EV, I added those into the currently active set. Also, multiply their respective probabilities by an exponential decay factor with a learning rate to penalise those strategies which had higher EVs. And then re norm the probabilities. For different random starting seeds this always rapidly (under 10 minutes of runtime on an M2 MacBook Air with 8 threads) converged into the ridge nadir I outlined above which strongly suggests it is the unique equilibrium.

I decided to polarise my approach and either go for solving very fast, or instead wait to submit until the last minute. The former could impress the judges if executed well. The latter means that obeying the

directive to repost will not tip off previously uninformed potential competitors to the prize. In the end, I was not fast, so I defaulted towards this latter route.

-

```
#include <algorithm>
#include <cmath>
#include <ctime>
#include <random>
#include <stdio.h>
#include <vector>
#include <memory>
#include <omp.h>
#include <string>
```

```
static const int N = 50;
static const int TOLL = 40;
static const int EW_ITERS = 1000;
static const int MAX_ACTIVE = 20000;
static const int SEED_A = 90;
static const int SEED_H = 5;
static const int MC_BASE = 1000;
static const int MC_BOOST = 10000;
static const int BOOST_MARK = 20;
static const int VS_GRIDS = 50000;
static const long double EXPAND_TOLERANCE = 0.0;
static const int NEDGE = 2 * N * (N + 1);
```

```
static const int NEW_CHECK = 50; // Global random samples per iteration
```

```
static const int N_H_STRAT = 42;
static const int MIN_CAP = 5;
static const int MAX_CAP = 30;
static const int NUM_CAPS = MAX_CAP - MIN_CAP + 1; // 26
```

```
static int H_VALS[N_H_STRAT]; // Other version checked that > 44 is Silly
```

```
void build_h() {
    int i = 0;
    for (int h = 5; h <= 30; ++h) H_VALS[i++] = h;
    for (int h = 45; h <= 60; ++h) H_VALS[i++] = h;
}
```

```
// Exact Joint Probability Distributions
```

```

double get_multinomial(int a, int kR, int kU) {
    if (kR + kU > 2 * a) return 0.0;
    if (a == 0) {
        return (kR == 0 && kU == 0) ? 1.0 : 0.0;
    }
    int n = 2 * a;
    double p = 1.0 / NEDGE;

    double log_prob = std::lgamma(n + 1)
        - std::lgamma(kR + 1)
        - std::lgamma(kU + 1)
        - std::lgamma(n - kR - kU + 1)
        + (kR + kU) * std::log(p)
        + (n - kR - kU) * std::log(1.0 - 2.0 * p);
    return std::exp(log_prob);
}

```

```

double get_binomial(int a, int k) {
    if (k > 2 * a) return 0.0;
    if (a == 0) return (k == 0) ? 1.0 : 0.0;
    int n = 2 * a;
    double p = 1.0 / NEDGE;

    double log_prob = std::lgamma(n + 1)
        - std::lgamma(k + 1)
        - std::lgamma(n - k + 1)
        + k * std::log(p)
        + (n - k) * std::log(1.0 - p);
    return std::exp(log_prob);
}

```

// Memory block for a single 'a' value's physics

```

struct RouteTables {
    long double pR [NUM_CAPS][NUM_CAPS][N_H_STRAT];
    long double ccR[NUM_CAPS][NUM_CAPS][N_H_STRAT];
    long double ccU[NUM_CAPS][NUM_CAPS][N_H_STRAT];
    long double ec1[NUM_CAPS];
};

```

```

struct Strategy {
    int h_val;
    int h_idx;
    int a_val;
};

```



```

        if (bR && !bU) {
            prob_R_given_k = 1.0;
        } else if (!bR && bU) {
            prob_R_given_k = 0.0;
        } else {
            prob_R_given_k = 0.5;
        }

        pr += p_joint * prob_R_given_k;
        expected_cost_R += p_joint * prob_R_given_k * costR;
        expected_cost_U += p_joint * (1.0 - prob_R_given_k) * costU;
    }
}

tab.pR[cR_idx][cU_idx][s] = pr;

if (pr > 1e-15) {
    tab.ccR[cR_idx][cU_idx][s] = expected_cost_R / pr;
} else {
    tab.ccR[cR_idx][cU_idx][s] = static_cast<long double>(cR);
}

long double pu = 1.0 - pr;
if (pu > 1e-15) {
    tab.ccU[cR_idx][cU_idx][s] = expected_cost_U / pu;
} else {
    tab.ccU[cR_idx][cU_idx][s] = static_cast<long double>(cU);
}
}
}
}

inline int eid_R(int x, int y) { return y * N + x; }
inline int eid_U(int x, int y) { return N * (N + 1) + x * N + y; }
inline int nid(int x, int y) { return x * (N + 1) + y; }

struct SimState {
    int base[NEDGE];
    long double flow[NEDGE];
    long double pmass[MAX_ACTIVE][(N + 1) * (N + 1)];
    long double probe_mass [(N + 1) * (N + 1)];
    long double probe_score[(N + 1) * (N + 1)];
};

```

```

void simulate_grid(SimState& st,
    const std::vector<long double>& Pf,
    const std::vector<Strategy>& probes,
    int num_active,
    const std::vector<RouteTables>& tables,
    std::vector<long double>& local_ev_acc,
    std::mt19937_64& rng,
    std::uniform_int_distribution<int>& bdist) {

    for (int i = 0; i < NEDGE; ++i) {
        st.base[i] = bdist(rng);
        st.flow[i] = 0.0;
    }

    // Pass 1: Traffic is ONLY generated by active strategies
    for (int s = 0; s < num_active; ++s) {
        for (int v = 0; v < (N + 1) * (N + 1); ++v) {
            st.pmass[s][v] = 0.0;
        }
        st.pmass[s][nid(0, 0)] = Pf[s];
    }

    for (int d = 0; d < 2 * N; ++d) {
        int x0 = (d >= N) ? d - N : 0;
        int x1 = (d < N) ? d : N;

        for (int x = x0; x <= x1; ++x) {
            int y = d - x;
            bool cR = (x < N);
            bool cU = (y < N);
            int v = nid(x, y);

            if (cR && cU) {
                int ir = eid_R(x, y);
                int iu = eid_U(x, y);

                int r5 = st.base[ir] - MIN_CAP;
                int u5 = st.base[iu] - MIN_CAP;

                if (r5 >= 0 && r5 < NUM_CAPS && u5 >= 0 && u5 < NUM_CAPS) {
                    for (int s = 0; s < num_active; ++s) {
                        long double m = st.pmass[s][v];
                        if (m < 1e-20f) continue;

```

```

        long double pr = tables[probes[s].a_val].pR[r5][u5][probes[s].h_idx];
        long double mR = m * pr;
        long double mU = m * (1.0 - pr);

        st.flow[ir] += mR;
        st.flow[iu] += mU;
        st.pmass[s][nid(x + 1, y)] += mR;
        st.pmass[s][nid(x, y + 1)] += mU;
    }
}
} else if (cR) {
    int ir = eid_R(x, y);
    for (int s = 0; s < num_active; ++s) {
        long double m = st.pmass[s][v];
        if (m < 1e-20f) continue;
        st.flow[ir] += m;
        st.pmass[s][nid(x + 1, y)] += m;
    }
} else {
    int iu = eid_U(x, y);
    for (int s = 0; s < num_active; ++s) {
        long double m = st.pmass[s][v];
        if (m < 1e-20f) continue;
        st.flow[iu] += m;
        st.pmass[s][nid(x, y + 1)] += m;
    }
}
}
}
}

```

```

// Pass 2: Evaluate ALL probes
int K = probes.size();
for (int s = 0; s < K; ++s) {
    for (int v = 0; v < (N + 1) * (N + 1); ++v) {
        st.probe_mass[v] = 0.0;
        st.probe_score[v] = 0.0;
    }
    st.probe_mass[nid(0, 0)] = 1.0;

    int a_val = probes[s].a_val;
    int h_idx = probes[s].h_idx;
    const RouteTables& tab = tables[a_val];

```

```

for (int d = 0; d < 2 * N; ++d) {
    int x0 = (d >= N) ? d - N : 0;
    int x1 = (d < N) ? d : N;

    for (int x = x0; x <= x1; ++x) {
        int y = d - x;
        bool cR = (x < N);
        bool cU = (y < N);
        int v = nid(x, y);

        long double pm = st.probe_mass[v];
        if (pm < 1e-20f) continue;
        long double sc = st.probe_score[v];

        if (cR && cU) {
            int ir = eid_R(x, y);
            int iu = eid_U(x, y);

            int r5 = st.base[ir] - MIN_CAP;
            int u5 = st.base[iu] - MIN_CAP;

            if (r5 >= 0 && r5 < NUM_CAPS && u5 >= 0 && u5 < NUM_CAPS) {
                long double pr = tab.pR[r5][u5][h_idx];
                long double pu = 1.0 - pr;
                long double fR = st.flow[ir];
                long double fU = st.flow[iu];

                long double cgR = (1.0 + fR) * (1.0 + fR) * (1.0 + fR);
                long double cgU = (1.0 + fU) * (1.0 + fU) * (1.0 + fU);
                long double eR = tab.ccR[r5][u5][h_idx] * cgR;
                long double eU = tab.ccU[r5][u5][h_idx] * cgU;

                long double mR = pm * pr;
                long double mU = pm * pu;

                st.probe_mass[nid(x + 1, y)] += mR;
                st.probe_score[nid(x + 1, y)] += sc * pr + mR * eR;
                st.probe_mass[nid(x, y + 1)] += mU;
                st.probe_score[nid(x, y + 1)] += sc * pu + mU * eU;
            }
        } else if (cR) {
            int ir = eid_R(x, y);
            int cap_idx = st.base[ir] - MIN_CAP;

```

```

        if (cap_idx >= 0 && cap_idx < NUM_CAPS) {
            long double fR = st.flow[ir];
            long double cgR = (1.0 + fR) * (1.0 + fR) * (1.0 + fR);
            long double ec = tab.ec1[cap_idx] * cgR;

            st.probe_mass[nid(x + 1, y)] += pm;
            st.probe_score[nid(x + 1, y)] += sc + pm * ec;
        }
    } else {
        int iu = eid_U(x, y);
        int cap_idx = st.base[iu] - MIN_CAP;

        if (cap_idx >= 0 && cap_idx < NUM_CAPS) {
            long double fU = st.flow[iu];
            long double cgU = (1.0 + fU) * (1.0 + fU) * (1.0 + fU);
            long double ec = tab.ec1[cap_idx] * cgU;

            st.probe_mass[nid(x, y + 1)] += pm;
            st.probe_score[nid(x, y + 1)] += sc + pm * ec;
        }
    }
}

long double dest_m = st.probe_mass[nid(N, N)];
if (dest_m > 1e-10f) {
    local_ev_acc[s] += static_cast<long double>(st.probe_score[nid(N, N)] / dest_m);
}
}

int main() {
    build_h();
    int nthreads = 1;
#ifdef _OPENMP
    nthreads = omp_get_max_threads();
#endif

    printf("Precomputing exact multinomial physics tables for a in [0, 300]... \n");
    fflush(stdout);
    std::vector<RouteTables> all_tables(301);

    #pragma omp parallel for
    for (int a = 0; a <= 300; ++a) {

```

```

    build_tables_for_a(a, all_tables[a]);
}

printf("Tables generated. Initializing Engine with Global Search...\n\n");
fflush(stdout);

int seed_h_idx = -1;
for (int i = 0; i < N_H_STRAT; ++i) {
    if (H_VALS[i] == SEED_H) {
        seed_h_idx = i;
        break;
    }
}
if (seed_h_idx == -1) return 1;

std::vector<Strategy> active_strats = {{SEED_H, seed_h_idx, SEED_A}};
std::vector<long double> P = {1.0};
std::vector<long double> ema = {0.0};

std::vector<std::vector<bool>> is_active(301, std::vector<bool>(N_H_STRAT, false));
is_active[SEED_A][seed_h_idx] = true;

std::vector<std::unique_ptr<SimState>> states(nthreads);
for (int t = 0; t < nthreads; ++t) {
    states[t] = std::make_unique<SimState>();
}

// Generator for global random scouting
std::mt19937 rng_scout(SEED_A + SEED_H + 1337);
std::uniform_int_distribution<int> dist_a(0, 300);
// Indices 0-25 correspond to h = 5 to 30 in H_VALS
std::uniform_int_distribution<int> dist_h_idx(0, 25);

for (int iter = 0; iter < EW_ITERS; ++iter) {
    long double t0 = omp_get_wtime();
    int current_mc = (iter >= EW_ITERS - BOOST_MARK) ? MC_BOOST : MC_BASE;

    // --- STEP 1: Scout valid neighbors + Global randoms ---
    std::vector<Strategy> neighbors;
    std::vector<std::vector<bool>> is_scouted(301, std::vector<bool>(N_H_STRAT, false));
    int num_active = active_strats.size();

    if (num_active < MAX_ACTIVE) {
        // Local flood-fill

```

```

const int da[] = {-1, 1, 0, 0};
const int dh[] = {0, 0, -1, 1};

for (const auto& strat : active_strats) {
    for (int i = 0; i < 4; ++i) {
        int na = strat.a_val + da[i];
        int nh = strat.h_idx + dh[i];

        if (na >= 0 && na <= 300 && nh >= 0 && nh < N_H_STRAT) {
            if (!is_active[na][nh] && !is_scouted[na][nh]) {
                neighbors.push_back({H_VALS[nh], nh, na});
                is_scouted[na][nh] = true;
            }
        }
    }
}

// Global random sampling
for (int i = 0; i < NEW_CHECK; ++i) {
    int ra = dist_a(rng_scout);
    int rh = dist_h_idx(rng_scout);

    if (!is_active[ra][rh] && !is_scouted[ra][rh]) {
        neighbors.push_back({H_VALS[rh], rh, ra});
        is_scouted[ra][rh] = true;
    }
}

std::vector<Strategy> probes = active_strats;
probes.insert(probes.end(), neighbors.begin(), neighbors.end());
int num_probes = probes.size();

std::vector<long double> ev_acc(num_probes, 0.0);
std::vector<long double> Pf(num_active);
for (int s = 0; s < num_active; ++s) {
    Pf[s] = static_cast<long double>(P[s]);
}

// --- STEP 2: Monte Carlo Simulation ---
#pragma omp parallel
{
    std::vector<long double> local_ev_acc(num_probes, 0.0);
    int tid = 0;

```

```

#ifdef _OPENMP
    tid = omp_get_thread_num();
#endif
    SimState& st = *states[tid];
    std::mt19937_64 rng(42 + static_cast<uint64_t>(iter) * 10000 + tid);
    std::uniform_int_distribution<int> bdist(MIN_CAP, MAX_CAP);

    #pragma omp for schedule(dynamic, 50)
    for (int mc = 0; mc < current_mc; ++mc) {
        simulate_grid(st, Pf, probes, num_active, all_tables, local_ev_acc, rng, bdist);
    }

    #pragma omp critical
    {
        for (int s = 0; s < num_probes; ++s) {
            ev_acc[s] += local_ev_acc[s];
        }
    }
}

// --- STEP 3: Evaluation & Active Set Expansion ---
std::vector<long double> ev(num_probes);
long double min_active_ev = 1e18;

for (int s = 0; s < num_probes; ++s) {
    ev[s] = (ev_acc[s] / current_mc) - probes[s].a_val;
    if (s < num_active && ev[s] < min_active_ev) {
        min_active_ev = ev[s];
    }
}

int added = 0;
for (int s = num_active; s < num_probes; ++s) {
    if (active_strats.size() >= MAX_ACTIVE) break;
    if (ev[s] <= min_active_ev + EXPAND_TOLERANCE) {
        active_strats.push_back(probes[s]);
        is_active[probes[s].a_val][probes[s].h_idx] = true;
        P.push_back(0.003);
        ema.push_back(ev[s]);
        added++;
    }
}

// --- STEP 4: Exponential Weights Update ---

```

```

int old_num_active = num_active;
num_active = active_strats.size();

long double ema_min = 1e18, ema_max = -1e18;
for (int s = 0; s < num_active; ++s) {
    if (s < old_num_active) {
        ema[s] = (iter == 0) ? ev[s] : (0.9 * ev[s] + 0.1 * ema[s]);
    }
    if (ema[s] < ema_min) ema_min = ema[s];
    if (ema[s] > ema_max) ema_max = ema[s];
}

long double lr = 0.25 / std::sqrt(1.0 + iter);
long double sum_w = 0.0;
std::vector<long double> w(num_active);

for (int s = 0; s < num_active; ++s) {
    w[s] = P[s] * exp(-lr * (ema[s] - ema_min));
    sum_w += w[s];
}

for (int s = 0; s < num_active; ++s) {
    P[s] = w[s] / sum_w;
}

printf("Iter %2d/%d [Active: %d | Promoted: %d | ema_min=%7.2f] time=%.1fs\n",
    iter + 1, EW_ITERS, static_cast<int>(active_strats.size()), added, ema_min, omp_get_wtime() -
t0);
fflush(stdout);
}

// --- PHASE 2: Final High-Precision Evaluation ---
int final_k = active_strats.size();
printf("\nRunning Final Evaluation on %d survived strategies (%d grids)...\n", final_k, VS_GRIDS);

std::vector<long double> fAvg_acc(final_k, 0.0);
std::vector<long double> Pfinal(final_k);
for (int s = 0; s < final_k; ++s) {
    Pfinal[s] = static_cast<long double>(P[s]);
}

#pragma omp parallel
{
    std::vector<long double> local_fAvg(final_k, 0.0);

```

```

    int tid = 0;
#ifdef _OPENMP
    tid = omp_get_thread_num();
#endif
    SimState& st = *states[tid];
    std::mt19937_64 rng(99999 + tid);
    std::uniform_int_distribution<int> bdist(MIN_CAP, MAX_CAP);

    #pragma omp for schedule(dynamic, 50)
    for (int mc = 0; mc < VS_GRIDS; ++mc) {
        simulate_grid(st, Pfinal, active_strats, final_k, all_tables, local_fAvg, rng, bdist);
    }

    #pragma omp critical
    {
        for (int s = 0; s < final_k; ++s) {
            fAvg_acc[s] += local_fAvg[s];
        }
    }
}

std::vector<long double> fAvg(final_k);
long double fmin = 1e18;
for (int s = 0; s < final_k; ++s) {
    fAvg[s] = (fAvg_acc[s] / VS_GRIDS) - active_strats[s].a_val;
    if (fAvg[s] < fmin) fmin = fAvg[s];
}

// --- Output Results ---
printf("\n=== Nash Equilibrium Distribution P(a, h) ===\n");
printf("%-5s %-7s %-10s %-10s %-10s\n", "a", "h", "P", "EV", "Regret");
printf("-----\n");

for (int s = 0; s < final_k; ++s) {
    if (1) {
        std::string label = (active_strats[s].h_val == 30) ? "30-44" : std::to_string(active_strats[s].h_val);
        printf("%-5d %-7s %.6f %10.4f %10.4f\n",
            active_strats[s].a_val, label.c_str(), P[s], fAvg[s], fAvg[s] - fmin);
    }
}

FILE* f = fopen("nash_floodfill_result.csv", "w");
if (f) {
    fprintf(f, "a,h,prob,ev,regret\n");
}

```

```

        for (int s = 0; s < final_k; ++s) {
            std::string label = (active_strats[s].h_val == 30) ? "30-44" : std::to_string(active_strats[s].h_val);
            fprintf(f, "%d,%s,%.8f,%.4f,%.4f\n",
                    active_strats[s].a_val, label.c_str(), P[s], fAvg[s], fAvg[s] - fmin);
        }
        fclose(f);
    }

    return 0;
}

```

-

```

"""
Nash Equilibrium Heatmap Plotter
Usage: python nash_heatmaps.py [path/to/result.csv]
Defaults to nash_floodfill_result.csv in the current directory.
"""

```

```

import sys
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.colors as mcolors
from matplotlib.ticker import MaxNLocator

```

— Config

```

CSV_PATH = sys.argv[1] if len(sys.argv) > 1 else "nash_floodfill_result.csv"
MIN_PROB_TO_PLOT = 1e-3 # rows below this prob are shown as "unexplored" grey
H_LABEL_30 = "30-44"    # display label for h=30 (the collapsed bin)

```

— Load

```

df = pd.read_csv(CSV_PATH)
df.columns = df.columns.str.strip()

```

```

# Keep only rows that were actually evaluated (prob > threshold OR regret is finite)
evaluated = df[df["prob"] > MIN_PROB_TO_PLOT].copy()

```

```

# For regret map we want ALL evaluated rows regardless of prob
all_eval = df[df["regret"].notna() & np.isfinite(df["regret"])].copy()

```

```

# Filter out rows that were never really explored (very high regret, zero prob)
# — use rows that the solver actually visited (prob > 0 OR regret < some cutoff)
all_eval = df[(df["prob"] > MIN_PROB_TO_PLOT) |
              (df["regret"] < df["regret"].quantile(0.95))].copy()

def make_pivot(data, value_col, a_col="a", h_col="h"):
    """Pivot to 2-D grid: rows=h (ascending), cols=a (ascending)."""
    pivot = data.pivot_table(index=h_col, columns=a_col, values=value_col, aggfunc="mean")
    pivot = pivot.sort_index(ascending=True)      # h low→high bottom→top
    pivot = pivot.sort_index(axis=1, ascending=True) # a left→right
    return pivot

def h_ticklabels(h_values):
    """Replace 30 with '30-44' for display."""
    def fmt(h):
        try:
            return H_LABEL_30 if int(float(h)) == 30 else str(int(float(h)))
        except (ValueError, TypeError):
            return str(h)
    return [fmt(h) for h in h_values]

def a_ticklabels(a_values, max_labels=15):
    """Thin out x-axis labels so they don't overlap."""
    n = len(a_values)
    step = max(1, n // max_labels)
    labels = []
    for i, a in enumerate(a_values):
        labels.append(str(int(a)) if i % step == 0 else "")
    return labels

# — Colormaps


---


# Probability: white → deep purple
prob_cmap = mcolors.LinearSegmentedColormap.from_list(
    "prob_purple",
    ["#eedf8", "#a09be0", "#5349b3", "#261e68"],
)
prob_cmap.set_bad(color="#e8e8e4") # grey for NaN / unexplored

# Regret: teal (low) → light grey (mid) → coral (high)
regret_cmap = mcolors.LinearSegmentedColormap.from_list(
    "regret_tealcoral",
    ["#1d9e75", "#a8a89a", "#d85a30"],
)

```

```
regret_cmap.set_bad(color="#e8e8e4")
```

```
# — Plot helper
```

```
def plot_heatmap(pivot, cmap, title, cbar_label, power=0.45, fig_scale=1.0):
```

```
    """
```

```
    Draw a heatmap from a pivot table.
```

```
    power: gamma for colour scaling (< 1 pulls out low values).
```

```
    """
```

```
    data = pivot.values.astype(float)
```

```
    # Mask NaNs (unexplored)
```

```
    masked = np.ma.masked_invalid(data)
```

```
    vmin = np.nanmin(data[np.isfinite(data)])
```

```
    vmax = np.nanmax(data[np.isfinite(data)])
```

```
    # Gamma-normalise
```

```
    norm = mcolors.PowerNorm(gamma=power, vmin=vmin, vmax=vmax)
```

```
    n_h, n_a = pivot.shape
```

```
    fig_w = max(8, n_a * 0.18 * fig_scale)
```

```
    fig_h = max(4, n_h * 0.45 * fig_scale)
```

```
    fig, ax = plt.subplots(figsize=(fig_w, fig_h))
```

```
    im = ax.imshow(
```

```
        masked,
```

```
        cmap=cmap,
```

```
        norm=norm,
```

```
        aspect="auto",
```

```
        origin="lower",      # h increases upward
```

```
        interpolation="nearest",
```

```
    )
```

```
    # Axes ticks
```

```
    a_vals = list(pivot.columns)
```

```
    h_vals = list(pivot.index)
```

```
    ax.set_xticks(range(n_a))
```

```
    ax.set_xticklabels(a_ticklabels(a_vals), fontsize=8, rotation=90)
```

```
    ax.set_yticks(range(n_h))
```

```
    ax.set_yticklabels(h_ticklabels(h_vals), fontsize=8)
```

```
    ax.set_xlabel("a (rebate)", fontsize=11, labelpad=6)
```

```

ax.set_ylabel("h (threshold)", fontsize=11, labelpad=6)
ax.set_title(title, fontsize=13, pad=10)

# Colourbar
cbar = fig.colorbar(im, ax=ax, fraction=0.03, pad=0.02)
cbar.set_label(cbar_label, fontsize=9)
cbar.ax.tick_params(labelsize=8)

# Thin grid lines between cells
ax.set_xticks(np.arange(-0.5, n_a, 1), minor=True)
ax.set_yticks(np.arange(-0.5, n_h, 1), minor=True)
ax.grid(which="minor", color="white", linewidth=0.3, alpha=0.4)
ax.tick_params(which="minor", bottom=False, left=False)

fig.tight_layout()
return fig

# — Build pivots

```

```

# Probability pivot — only strategies with meaningful mass
prob_pivot = make_pivot(evaluated, "prob")

# Regret pivot — all evaluated strategies (mask zeros for log/power norm)
regret_pivot = make_pivot(all_eval, "regret")

# — Draw

```

```

fig1 = plot_heatmap(
    prob_pivot,
    cmap=prob_cmap,
    title="Nash Equilibrium —  $P(a, h)$ ",
    cbar_label="Probability",
    power=0.4,
)

fig2 = plot_heatmap(
    regret_pivot,
    cmap=regret_cmap,
    title="Nash Equilibrium —  $\text{Regret}(a, h)$ ",
    cbar_label="Regret",
    power=0.5,
)

```

```
fig1.savefig("nash_prob_heatmap.png", dpi=150, bbox_inches="tight")
fig2.savefig("nash_regret_heatmap.png", dpi=150, bbox_inches="tight")
print("Saved: nash_prob_heatmap.png, nash_regret_heatmap.png")
```

```
plt.show()
```

```
-
```

```
/*
 * Nash Equilibrium Solver: Best-Response Dynamics on Fixed Grid Sample
 *
 * Method:
 * 1. Load top-K strategies by probability from a CSV file.
 * 2. Sample M grids (random base capacities) once and fix them.
 * 3. Iterate best-response dynamics:
 * - Compute EV(s, P) for every strategy s under current mix P
 * using the fixed grid set.
 * - Find best response (min EV strategy).
 * - Mix:  $P \leftarrow (1 - \alpha) * P + \alpha * e_{\text{best\_response}}$ 
 * where  $\alpha = 1 / (\text{iter} + 1)$  [equal-weight averaging]
 * - Repeat until  $\text{max\_regret} < \text{EPSILON}$ .
 *
 * Compile:
 * g++ -O3 -fopenmp -std=c++17 -o nash_br_solver nash_br_solver.cpp
 *
 * Run:
 * ./nash_br_solver [path/to/nash_floodfill_result.csv]
 * Defaults to ./nash_floodfill_result.csv
 */
```

```
#include <algorithm>
#include <cassert>
#include <cmath>
#include <cstdio>
#include <cstring>
#include <fstream>
#include <memory>
#include <numeric>
#include <random>
#include <sstream>
#include <string>
#include <vector>
#include <omp.h>
```

```
// — Game constants (must match the original solver) —————
static const int   N       = 50;
static const int   TOLL    = 40;
static const int   MIN_CAP = 5;
static const int   MAX_CAP = 30;
static const int   NUM_CAPS = MAX_CAP - MIN_CAP + 1; // 26
static const int   NEDGE   = 2 * N * (N + 1);      // 5100
static const int   NNODES  = (N + 1) * (N + 1);    // 2601
```

```
// — Solver parameters
```

```
static const int   TOP_K   = 20;    // strategies to load from CSV
static const int   M_GRIDS = 40000; // fixed grid sample size
static const int   MAX_ITER = 50000; // iteration cap
static const long double EPSILON = 0.01; // convergence: max regret < EPSILON
static const int   PRINT_EVERY = 40;
```

```
// — Grid helpers
```

```
inline int eid_R(int x, int y) { return y * N + x; }
inline int eid_U(int x, int y) { return N * (N + 1) + x * N + y; }
inline int nid (int x, int y) { return x * (N + 1) + y; }
```

```
// — Exact Joint Probability Distributions —————
static double get_multinomial(int a, int kR, int kU) {
    if (kR + kU > 2 * a) return 0.0;
    if (a == 0) {
        return (kR == 0 && kU == 0) ? 1.0 : 0.0;
    }
    int n = 2 * a;
    double p = 1.0 / NEDGE;

    double log_prob = std::lgamma(n + 1)
        - std::lgamma(kR + 1)
        - std::lgamma(kU + 1)
        - std::lgamma(n - kR - kU + 1)
        + (kR + kU) * std::log(p)
        + (n - kR - kU) * std::log(1.0 - 2.0 * p);
    return std::exp(log_prob);
}
```

```
static double get_binomial(int a, int k) {
    if (k > 2 * a) return 0.0;
    if (a == 0) return (k == 0) ? 1.0 : 0.0;
```

```

int n = 2 * a;
double p = 1.0 / NEDGE;

double log_prob = std::lgamma(n + 1)
    - std::lgamma(k + 1)
    - std::lgamma(n - k + 1)
    + k * std::log(p)
    + (n - k) * std::log(1.0 - p);
return std::exp(log_prob);
}

```

// — Route tables

```

struct RouteTables {
    long double pR [NUM_CAPS][NUM_CAPS]; // routing prob right, per (cR_idx, cU_idx)
    long double ccR[NUM_CAPS][NUM_CAPS]; // E[private cost | go right]
    long double ccU[NUM_CAPS][NUM_CAPS]; // E[private cost | go up]
    long double ec1[NUM_CAPS];           // E[cost | single forced edge]
};

static void build_tables(int a, int h, RouteTables& tab) {
    // 1. Calculate Expected Cost (ec1) using Exact Binomial Distribution
    for (int ci = 0; ci < NUM_CAPS; ++ci) {
        int c = ci + MIN_CAP;
        long double expected_cost = 0.0;
        long double weight_sum = 0.0;

        for (int k = 0; k <= std::min(40, 2 * a); ++k) {
            long double p = get_binomial(a, k);
            if (p < 1e-15) continue;

            expected_cost += p * (c + TOLL * k);
            weight_sum += p;
        }
        tab.ec1[ci] = (weight_sum > 1e-15) ? (expected_cost / weight_sum) : static_cast<long double>(c);
    }

    // 2. Calculate Exact Joint Probabilities and Edge Conditions using Multinomial
    for (int ri = 0; ri < NUM_CAPS; ++ri) {
        for (int ui = 0; ui < NUM_CAPS; ++ui) {
            int cR = ri + MIN_CAP;
            int cU = ui + MIN_CAP;

            long double pr = 0.0;

```

```

long double expected_cost_R = 0.0;
long double expected_cost_U = 0.0;

for (int kR = 0; kR <= std::min(40, 2 * a); ++kR) {
    for (int kU = 0; kU <= std::min(40, 2 * a - kR); ++kU) {
        long double p_joint = get_multinomial(a, kR, kU);
        if (p_joint < 1e-15) continue;

        long double costR = cR + TOLL * kR;
        long double costU = cU + TOLL * kU;

        bool bR = (costR <= h);
        bool bU = (costU <= h);

        long double prob_R_given_k = 0.0;
        if (bR && !bU) {
            prob_R_given_k = 1.0;
        } else if (!bR && bU) {
            prob_R_given_k = 0.0;
        } else {
            prob_R_given_k = 0.5;
        }

        pr += p_joint * prob_R_given_k;
        expected_cost_R += p_joint * prob_R_given_k * costR;
        expected_cost_U += p_joint * (1.0 - prob_R_given_k) * costU;
    }
}

tab.pR[ri][ui] = static_cast<long double>(pr);

if (pr > 1e-15) {
    tab.ccR[ri][ui] = static_cast<long double>(expected_cost_R / pr);
} else {
    tab.ccR[ri][ui] = static_cast<long double>(cR);
}

long double pu = 1.0 - pr;
if (pu > 1e-15) {
    tab.ccU[ri][ui] = static_cast<long double>(expected_cost_U / pu);
} else {
    tab.ccU[ri][ui] = static_cast<long double>(cU);
}
}

```

```
}  
}
```

```
// — Strategy
```

```
struct Strategy {  
    int  a_val;  
    int  h_val;  
    long double csv_prob; // original probability from CSV (for reference)  
};
```

```
// — Fixed grid sample
```

```
// Each grid is just a flat array of base capacities for each edge.
```

```
struct Grid {  
    int base[NEDGE];  
};
```

```
// — Per-thread workspace
```

```
struct Workspace {  
    long double flow [NEDGE];  
    long double pmass[NNODES]; // reused per strategy in pass 1  
    long double probe_mass [NNODES];  
    long double probe_score[NNODES];  
};
```

```
// — Evaluate all strategies on one fixed grid
```

```
//
```

```
// Pass 1: build the flow field from the population mix P (weights Pf[s]).
```

```
//     Only strategies with Pf[s] > 0 contribute.
```

```
// Pass 2: for each strategy s, walk the probe through the flow field and
```

```
//     accumulate expected cost.
```

```
//
```

```
static void eval_grid(  
    const Grid&                grid,  
    const std::vector<long double>& Pf,      // population weights  
    const std::vector<Strategy>& strats,  
    const std::vector<RouteTables>& tabs,    // one per strategy  
    std::vector<long double>& ev_acc,      // accumulated EV per strategy  
    Workspace&                  ws)  
{  
    int K = (int)strats.size();
```

// — Pass 1: flow field

```
std::memset(ws.flow, 0, sizeof(long double) * NEDGE);
```

```
for (int s = 0; s < K; ++s) {  
    if (Pf[s] < 1e-20f) continue;  
    std::memset(ws.pmass, 0, sizeof(long double) * NNODES);  
    ws.pmass[nid(0,0)] = Pf[s];
```

```
    for (int d = 0; d < 2 * N; ++d) {  
        int x0 = (d >= N) ? d - N : 0;  
        int x1 = (d < N) ? d : N;  
        for (int x = x0; x <= x1; ++x) {  
            int y = d - x;  
            bool cR = (x < N), cU = (y < N);  
            int v = nid(x, y);  
            long double m = ws.pmass[v];  
            if (m < 1e-20f) continue;  
  
            if (cR && cU) {  
                int ir = eid_R(x, y), iu = eid_U(x, y);  
                int ri = grid.base[ir] - MIN_CAP;  
                int ui = grid.base[iu] - MIN_CAP;  
                long double pr = tabs[s].pR[ri][ui];  
                ws.flow[ir] += m * pr;  
                ws.flow[iu] += m * (1.0f - pr);  
                ws.pmass[nid(x+1, y)] += m * pr;  
                ws.pmass[nid(x, y+1)] += m * (1.0f - pr);  
            } else if (cR) {  
                ws.flow[eid_R(x,y)] += m;  
                ws.pmass[nid(x+1, y)] += m;  
            } else {  
                ws.flow[eid_U(x,y)] += m;  
                ws.pmass[nid(x, y+1)] += m;  
            }  
        }  
    }  
}
```

// — Pass 2: evaluate each strategy as a probe

```
for (int s = 0; s < K; ++s) {  
    std::memset(ws.probe_mass, 0, sizeof(long double) * NNODES);  
    std::memset(ws.probe_score, 0, sizeof(long double) * NNODES);  
    ws.probe_mass[nid(0,0)] = 1.0f;
```

```

const RouteTables& tab = tabs[s];

for (int d = 0; d < 2 * N; ++d) {
    int x0 = (d >= N) ? d - N : 0;
    int x1 = (d < N) ? d : N;
    for (int x = x0; x <= x1; ++x) {
        int y = d - x;
        bool cR = (x < N), cU = (y < N);
        int v = nid(x, y);
        long double pm = ws.probe_mass[v];
        if (pm < 1e-20f) continue;
        long double sc = ws.probe_score[v];

        if (cR && cU) {
            int ir = eid_R(x, y), iu = eid_U(x, y);
            int ri = grid.base[ir] - MIN_CAP;
            int ui = grid.base[iu] - MIN_CAP;

            long double pr = tab.pR[ri][ui];
            long double pu = 1.0f - pr;
            long double fR = ws.flow[ir];
            long double fU = ws.flow[iu];
            long double cgR = (1.0f + fR) * (1.0f + fR) * (1.0f + fR);
            long double cgU = (1.0f + fU) * (1.0f + fU) * (1.0f + fU);
            long double eR = tab.ccR[ri][ui] * cgR;
            long double eU = tab.ccU[ri][ui] * cgU;

            long double mR = pm * pr, mU = pm * pu;
            ws.probe_mass[nid(x+1, y)] += mR;
            ws.probe_score[nid(x+1, y)] += sc * pr + mR * eR;
            ws.probe_mass[nid(x, y+1)] += mU;
            ws.probe_score[nid(x, y+1)] += sc * pu + mU * eU;
        } else if (cR) {
            int ir = eid_R(x, y);
            int ci = grid.base[ir] - MIN_CAP;
            long double cg = (1.0f + ws.flow[ir]);
            cg = cg * cg * cg;
            long double ec = tab.ec1[ci] * cg;
            ws.probe_mass[nid(x+1, y)] += pm;
            ws.probe_score[nid(x+1, y)] += sc + pm * ec;
        } else {
            int iu = eid_U(x, y);
            int ci = grid.base[iu] - MIN_CAP;

```

```

        long double cg = (1.0f + ws.flow[iu]);
        cg = cg * cg * cg;
        long double ec = tab.ec1[ci] * cg;
        ws.probe_mass[nid(x, y+1)] += pm;
        ws.probe_score[nid(x, y+1)] += sc + pm * ec;
    }
}

long double dest_m = ws.probe_mass[nid(N, N)];
if (dest_m > 1e-10f)
    ev_acc[s] += static_cast<long double>(ws.probe_score[nid(N,N)] / dest_m);
}
}

```

// — CSV loading

```

static std::vector<Strategy> load_top_k(const std::string& path, int k) {
    std::ifstream f(path);
    if (!f.is_open()) {
        fprintf(stderr, "Cannot open CSV: %s\n", path.c_str());
        std::exit(1);
    }

    std::string line;
    std::getline(f, line); // header

    struct Row { int a, h; long double prob; };
    std::vector<Row> rows;

    while (std::getline(f, line)) {
        if (line.empty()) continue;
        std::istringstream ss(line);
        std::string sa, sh, sprob, sev, sreg;
        if (!std::getline(ss, sa, ',')) continue;
        if (!std::getline(ss, sh, ',')) continue;
        if (!std::getline(ss, sprob, ',')) continue;

        int a = std::stoi(sa);
        long double prob = std::stod(sprob);

        // h may be "30-44" — treat as 30
        int h;
        if (sh.find('-') != std::string::npos) h = 30;
    }
}

```

```

        else h = std::stoi(sh);

        rows.push_back({a, h, prob});
    }

    // Sort descending by prob
    std::sort(rows.begin(), rows.end(),
        [](const Row& x, const Row& y){ return x.prob > y.prob; });

    int take = std::min(k, (int)rows.size());
    std::vector<Strategy> result;
    result.reserve(take);
    for (int i = 0; i < take; ++i)
        result.push_back({rows[i].a, rows[i].h, rows[i].prob});

    return result;
}

// — Main


---


int main(int argc, char* argv[]) {
    const std::string csv_path = (argc > 1) ? argv[1] : "./nash_floodfill_result.csv";

    // — Load strategies
    

---


    printf("Loading top %d strategies from %s ...\n", TOP_K, csv_path.c_str());
    std::vector<Strategy> strats = load_top_k(csv_path, TOP_K);

    // Manually filter to exactly the strategies we want
    // This can be extended further to verify that stuff like {0,10} or {1,11} {1,12}... {299,30} kind of
    // stuff is not good.
    std::vector<std::pair<int,int>> whitelist = {
        {300,30},{300,29},{300,28},{300,27},{300,26},
        {0,21},{0,20},{0,19},{0,18},{0,17},{0,16},{0,15},{0,14},{0,13},{0,12},{0,11}
        ,{59,22},{60,22},{61,22},{62,22},{140,23},{141,23},{142,23},{143,23},{144,23},{214,24},{215,24},{216,24},{
        217,24},{218,24},{219,24},{220,24},{290,25},{291,25},{292,25},{293,25}
    };

    std::vector<Strategy> filtered;
    for (auto& [wa, wh] : whitelist) {
        // find in loaded strats
        bool found = false;

```

```

for (auto& s : strats) {
    if (s.a_val == wa && s.h_val == wh) {
        filtered.push_back(s);
        found = true;
        break;
    }
}
if (!found) {
    // not in CSV — add with zero csv_prob
    filtered.push_back({wa, wh, 0.0});
}
}

strats = filtered;

int K = (int)strats.size();
printf("Loaded %d strategies:\n", K);
printf(" %-6s %-6s %-12s\n", "a", "h", "csv_prob");
for (int s = 0; s < K; ++s)
    printf(" %-6d %-6d %.8f\n", strats[s].a_val, strats[s].h_val, strats[s].csv_prob);

// — Build route tables



---



printf("\nBuilding route tables with exact distributions...\n");
std::vector<RouteTables> tabs(K);
#pragma omp parallel for schedule(dynamic)
for (int s = 0; s < K; ++s)
    build_tables(strats[s].a_val, strats[s].h_val, tabs[s]);

// — Sample M fixed grids



---



printf("Sampling %d fixed grids...\n", M_GRIDS);
std::vector<Grid> grids(M_GRIDS);
{
    std::mt19937 rng(739572);
    std::uniform_int_distribution<int> cap_dist(MIN_CAP, MAX_CAP);
    for (int m = 0; m < M_GRIDS; ++m)
        for (int e = 0; e < NEDGE; ++e)
            grids[m].base[e] = cap_dist(rng);
}

// — Initialize P from CSV probabilities (renormalized over top-K) —————
std::vector<long double> P(K);
{

```

```

    long double sum = 0;
    for (int s = 0; s < K; ++s) sum += strats[s].csv_prob;
    for (int s = 0; s < K; ++s) P[s] = strats[s].csv_prob / sum;
}

```

```

// — Per-thread workspaces

```

```

int nthreads = 1;
#ifdef _OPENMP
nthreads = omp_get_max_threads();
#endif
std::vector<std::unique_ptr<Workspace>> workspaces(nthreads);
for (int t = 0; t < nthreads; ++t)
    workspaces[t] = std::make_unique<Workspace>();

printf("Running best-response dynamics (%d threads)...\n\n", nthreads);

```

```

// — Best-response dynamics

```

```

for (int iter = 0; iter < MAX_ITER; ++iter) {

    // Build long double population weights
    std::vector<long double> Pf(K);
    for (int s = 0; s < K; ++s) Pf[s] = static_cast<long double>(P[s]);

    // Accumulate EV over all M fixed grids in parallel
    std::vector<long double> ev_acc(K, 0.0);

    #pragma omp parallel
    {
        std::vector<long double> local_acc(K, 0.0);
        int tid = 0;
        #ifdef _OPENMP
        tid = omp_get_thread_num();
        #endif
        Workspace& ws = *workspaces[tid];

        #pragma omp for schedule(dynamic, 20)
        for (int m = 0; m < M_GRIDS; ++m)
            eval_grid(grids[m], Pf, strats, tabs, local_acc, ws);

        #pragma omp critical
        for (int s = 0; s < K; ++s)
            ev_acc[s] += local_acc[s];
    }
}

```

```

}

// EV = average cost - rebate a
std::vector<long double> ev(K);
for (int s = 0; s < K; ++s)
    ev[s] = ev_acc[s] / M_GRIDS - strats[s].a_val;

long double ev_min = *std::min_element(ev.begin(), ev.end());
long double ev_max = *std::max_element(ev.begin(), ev.end());
long double max_regret = ev_max - ev_min;

// Best response: strategy with minimum EV
int br = (int)(std::min_element(ev.begin(), ev.end()) - ev.begin());

// Equal-weight averaging step: alpha = 1 / (iter + 1)
long double alpha = 0.5 / (std::max(iter, 200) + 1);
for (int s = 0; s < K; ++s)
    P[s] = (1.0 - alpha) * P[s] + alpha * (s == br ? 1.0 : 0.0);

if (iter % PRINT_EVERY == 0 || max_regret < EPSILON) {
    printf("Iter %5d | BR=(%d,%d) | ev_min=%8.8f | max_regret=%7.8f | alpha=%8.8f\n",
        iter + 1,
        strats[br].a_val, strats[br].h_val,
        ev_min, max_regret, alpha);
    fflush(stdout);
}

if ((iter) % PRINT_EVERY == 0) {
    printf("\n P distribution at iter %d:\n", iter + 1);
    for (int s = 0; s < K; ++s) {
        if (1)
            printf("  a=%-4d h=%-4d P=%8.8f EV=%10.8f regret=%8.8f\n",
                strats[s].a_val, strats[s].h_val,
                P[s], ev[s], ev[s] - ev_min);
    }
    printf("\n");
    fflush(stdout);
}

if (max_regret < EPSILON) {
    printf("\nConverged at iteration %d (max_regret=%8.8f < %8.8f)\n",
        iter + 1, max_regret, EPSILON);
    break;
}

```

```
}
```

```
// — Final EV computation
```

```
printf("\nRunning final EV computation...\n");
std::vector<long double> Pf(K);
for (int s = 0; s < K; ++s) Pf[s] = static_cast<long double>(P[s]);
```

```
std::vector<long double> ev_acc(K, 0.0);
#pragma omp parallel
{
    std::vector<long double> local_acc(K, 0.0);
    int tid = 0;
    #ifdef _OPENMP
    tid = omp_get_thread_num();
    #endif
    Workspace& ws = *workspaces[tid];
    #pragma omp for schedule(dynamic, 20)
    for (int m = 0; m < M_GRIDS; ++m)
        eval_grid(grids[m], Pf, strats, tabs, local_acc, ws);
    #pragma omp critical
    for (int s = 0; s < K; ++s)
        ev_acc[s] += local_acc[s];
}
```

```
std::vector<long double> ev_final(K);
long double ev_min = 1e18;
for (int s = 0; s < K; ++s) {
    ev_final[s] = ev_acc[s] / M_GRIDS - strats[s].a_val;
    if (ev_final[s] < ev_min) ev_min = ev_final[s];
}
```

```
// — Output
```

```
printf("\n=== Fixed-Point Nash Equilibrium (top-%d strategies, %d grids) ===\n", K, M_GRIDS);
printf("%-6s %-6s %-12s %-12s %-10s\n", "a", "h", "P", "EV", "Regret");
printf("-----\n");
for (int s = 0; s < K; ++s) {
    if (P[s] > 1e-6) {
        printf("%-6d %-6d %.8f  %10.8f  %10.8f\n",
            strats[s].a_val, strats[s].h_val,
            P[s], ev_final[s], ev_final[s] - ev_min);
    }
}
```

```

// Write CSV
FILE* f = fopen("nash_br_result.csv", "w");
if (f) {
    fprintf(f, "a,h,prob,ev,regret\n");
    for (int s = 0; s < K; ++s) {
        fprintf(f, "%d,%d,%.8f,%.8f,%.8f\n",
            strats[s].a_val, strats[s].h_val,
            P[s], ev_final[s], ev_final[s] - ev_min);
    }
    fclose(f);
    printf("\nResults written to nash_br_result.csv\n");
}

return 0;
}

-

// =====
// The Congested Random Walk — Modern C++ CRN Equilibrium Solver
// -----
// Computes the symmetric mixed Nash equilibrium distribution P(h) over the
// threshold h using Time-Averaged No-Regret Learning and Common Random Numbers.
//
// Build: g++ -O3 -march=native -std=c++17 -pthread -o solver modern_solver.cpp
// Run: ./solver
// =====

#include <iostream>
#include <vector>
#include <array>
#include <cmath>
#include <numeric>
#include <future>
#include <algorithm>
#include <iomanip>

// =====
// CONFIGURATION
// =====

namespace Config {
    constexpr int GRID_N      = 50;
    constexpr int PLAYERS     = 8;

```

```

constexpr int NUM_EDGES    = 2 * GRID_N * (GRID_N + 1); // 5100 edges
constexpr int UP_EDGE_OFFSET = GRID_N * (GRID_N + 1);

constexpr int  ROUNDS      = 250;
constexpr int  REPS_PER_ROUND = 100000;
constexpr double LEARNING_RATE = 0.03;
constexpr int  BURN_IN     = 40;

constexpr int  FINAL_EVAL_MULT = 20;
}

// =====
// CORE COMPONENTS
// =====

// Precomputed congestion penalties:  $(1 + f)^3$  where  $f = \text{traffic} / 8$ 
const std::array<double, 9> CONGESTION_PENALTY = [] {
    std::array<double, 9> arr{};
    for (int c = 0; c <= 8; ++c) {
        double f = c / 8.0;
        arr[c] = (1.0 + f) * (1.0 + f) * (1.0 + f);
    }
    return arr;
}();

// Fast, thread-local PRNG utilizing Lemire's divisionless bounds
class FastRng {
    uint64_t s0, s1;
public:
    explicit FastRng(uint64_t seed) {
        s0 = seed ^ 0x9E3779B97F4A7C15ULL;
        s1 = seed * 0xBF58476D1CE4E5B9ULL + 1;
        if (!s0) s0 = 1;
        if (!s1) s1 = 2;
        next_u64(); next_u64();
    }

    inline uint64_t next_u64() {
        uint64_t x = s0, y = s1;
        s0 = y;
        x ^= x << 23;
        s1 = x ^ y ^ (x >> 17) ^ (y >> 26);
        return s1 + y;
    }
}

```

```

inline uint32_t next_u32() { return static_cast<uint32_t>(next_u64() >> 32); }
inline double next_double() { return (next_u64() >> 11) * (1.0 / 9007199254740992.0); }

// Lemire's trick: fast bounded integer without modulo division
inline int rand_int(int lo, int hi) {
    uint32_t range = static_cast<uint32_t>(hi - lo + 1);
    return lo + static_cast<int>((static_cast<uint64_t>(next_u32()) * range) >> 32);
}
};

// =====
// SIMULATION ENGINE
// =====

// Memory allocated exactly once per thread to maintain L1 cache locality
struct ThreadContext {
    std::vector<int> base_costs;
    std::vector<int> bg_traffic;
    std::vector<int> touched_edges;

    ThreadContext()
        : base_costs(Config::NUM_EDGES, 0),
          bg_traffic(Config::NUM_EDGES, 0) {
        touched_edges.reserve(Config::PLAYERS * 2 * Config::GRID_N + 10);
    }
};

class GameSimulator {
    std::vector<int> active_h_vals;

    inline int right_edge(int x, int y) const { return y * Config::GRID_N + x; }
    inline int up_edge(int x, int y) const { return Config::UP_EDGE_OFFSET + x * Config::GRID_N + y; }

    // Helper: Samples a background player's strategy from the CDF
    inline int sample_strategy(FastRng& rng, const std::vector<double>& cdf, const std::vector<int>&
active_idx) const {
        double u = rng.next_double();
        auto it = std::lower_bound(cdf.begin(), cdf.end(), u);
        int dist = std::distance(cdf.begin(), it);
        return active_idx[std::min(dist, static_cast<int>(cdf.size() - 1))];
    }

    // Branchless edge routing logic

```

```

inline void route_path(const std::vector<int>& base, int h, FastRng& rng,
                      std::vector<int>& bg, std::vector<int>* touched = nullptr, double* cost_out = nullptr)
const {
    int x = 0, y = 0;
    double path_cost = 0;

    while (x < Config::GRID_N || y < Config::GRID_N) {
        int e;
        bool can_r = (x < Config::GRID_N);
        bool can_u = (y < Config::GRID_N);

        if (can_r && !can_u) { e = right_edge(x, y); x++; }
        else if (can_u && !can_r) { e = up_edge(x, y); y++; }
        else {
            int er = right_edge(x, y);
            int eu = up_edge(x, y);

            bool ok_r = base[er] <= h;
            bool ok_u = base[eu] <= h;

            // Branchless evaluation: If both are ok or both are bad, flip a coin.
            bool go_r = (ok_r ^ ok_u) ? ok_r : (rng.next_u32() & 1u);

            if (go_r) { e = er; x++; } else { e = eu; y++; }
        }

        if (touched) {
            if (bg[e] == 0) touched->push_back(e);
            bg[e]++;
        }
        if (cost_out) {
            path_cost += base[e] * CONGESTION_PENALTY[bg[e] + 1]; // +1 for the focal player
        }
    }

    if (cost_out) *cost_out += path_cost;
}

public:
GameSimulator() {
    for (int h = 5; h <= 30; ++h) active_h_vals.push_back(h);
    for (int h = 45; h <= 60; ++h) active_h_vals.push_back(h);
}

```

```

const std::vector<int>& get_strategies() const { return active_h_vals; }

// The primary Monte Carlo worker task designed for std::async
std::vector<double> evaluate_crn_batch(const std::vector<double>& current_q, int reps, uint64_t
seed) const {
    FastRng rng(seed);
    ThreadContext ctx;
    int num_strats = active_h_vals.size();
    std::vector<double> sum_costs(num_strats, 0.0);

    // Build CDF for background players
    std::vector<int> active_idx;
    std::vector<double> cdf;
    double cumulative = 0.0;
    for (int i = 0; i < num_strats; ++i) {
        if (current_q[i] > 0) {
            cumulative += current_q[i];
            active_idx.push_back(i);
            cdf.push_back(cumulative);
        }
    }
    if (!cdf.empty()) cdf.back() = 1.0;

    for (int iter = 0; iter < reps; ++iter) {
        // 1. Generate CRN Base Grid
        for (int e = 0; e < Config::NUM_EDGES; ++e) {
            ctx.base_costs[e] = rng.rand_int(5, 30);
        }

        // 2. Clear previous traffic and route 7 Background Players
        for (int e : ctx.touched_edges) ctx.bg_traffic[e] = 0;
        ctx.touched_edges.clear();

        for (int p = 0; p < Config::PLAYERS - 1; ++p) {
            int h_idx = sample_strategy(rng, cdf, active_idx);
            route_path(ctx.base_costs, active_h_vals[h_idx], rng, ctx.bg_traffic, &ctx.touched_edges,
nullptr);
        }

        // 3. Evaluate focal candidates on the exact same CRN grid
        for (int i = 0; i < num_strats; ++i) {
            route_path(ctx.base_costs, active_h_vals[i], rng, ctx.bg_traffic, nullptr, &sum_costs[i]);
        }
    }
}

```

```

        // Convert sums to expected averages
        for (double& cost : sum_costs) cost /= static_cast<double>(reps);
        return sum_costs;
    }
};

// =====
// MAIN REPLICATOR DYNAMICS LOOP
// =====

int main() {
    GameSimulator sim;
    const auto& strategies = sim.get_strategies();
    int num_strats = strategies.size();

    int n_threads = std::thread::hardware_concurrency();
    if (n_threads <= 0) n_threads = 4;

    std::cout << "=====\n";
    std::cout << " Modern C++ CRN Equilibrium Solver\n";
    std::cout << " " << n_threads << " Threads | " << Config::REPS_PER_ROUND << " Sims/Round | " <<
    Config::ROUNDS << " Rounds\n";
    std::cout << "=====\n\n";

    // Initialize Q uniformly across the low band [5, 30]
    std::vector<double> Q(num_strats, 0.0);
    int low_band_count = std::count_if(strategies.begin(), strategies.end(), [](int h) { return h <= 30; });
    for (int i = 0; i < num_strats; ++i) {
        if (strategies[i] <= 30) Q[i] = 1.0 / low_band_count;
    }

    std::vector<double> Q_sum(num_strats, 0.0);
    int avg_count = 0;
    uint64_t master_seed = 42069;

    std::cout << std::left << std::setw(8) << "Round"
        << std::setw(12) << "Meta Cost"
        << std::setw(12) << "Best Pure"
        << "Exploitability" << "\n";
    std::cout << std::string(50, '-') << "\n";

    for (int round = 0; round < Config::ROUNDS; ++round) {

        // 1. Dispatch Multi-threaded Evaluation via std::async

```

```

std::vector<std::future<std::vector<double>>> futures;
int reps_per_thread = Config::REPS_PER_ROUND / n_threads;

for (int t = 0; t < n_threads; ++t) {
    int reps = (t == n_threads - 1) ? (Config::REPS_PER_ROUND - reps_per_thread * (n_threads - 1)) :
reps_per_thread;
    futures.push_back(std::async(std::launch::async, &GameSimulator::evaluate_crn_batch, &sim,
std::cref(Q), reps, master_seed + t));
}

// 2. Aggregate Results
std::vector<double> costs(num_strats, 0.0);
for (auto& fut : futures) {
    auto thread_costs = fut.get();
    for (int i = 0; i < num_strats; ++i) {
        // Weight the thread's contribution based on an equal split
        costs[i] += thread_costs[i] * (1.0 / n_threads);
    }
}
master_seed += n_threads;

// 3. Replicator Dynamics Math
double expected_meta_cost = std::inner_product(Q.begin(), Q.end(), costs.begin(), 0.0);
double best_pure_cost = *std::min_element(costs.begin(), costs.end());

for (int i = 0; i < num_strats; ++i) {
    Q[i] *= std::exp(-Config::LEARNING_RATE * (costs[i] - expected_meta_cost));
}

// Normalize Q
double sum_q = std::accumulate(Q.begin(), Q.end(), 0.0);
for (double& q : Q) q /= sum_q;

// 4. Time-Averaging (Burn-in handling)
if (round >= Config::BURN_IN) {
    for (int i = 0; i < num_strats; ++i) Q_sum[i] += Q[i];
    avg_count++;
}

if (round % 10 == 0 || round == Config::ROUNDS - 1) {
    std::cout << std::left << std::setw(8) << round
        << std::fixed << std::setprecision(2) << std::setw(12) << expected_meta_cost
        << std::setw(12) << best_pure_cost
        << (expected_meta_cost - best_pure_cost) << "\n";
}

```

```

    }
}

//
=====
// FINAL OUTPUT & VERIFICATION
//
=====

// Calculate final equilibrium from time-averaged sums
std::vector<double> Q_eq(num_strats, 0.0);
double final_sum = 0.0;
for (int i = 0; i < num_strats; ++i) {
    Q_eq[i] = Q_sum[i] / avg_count;
    if (Q_eq[i] < 1e-4) Q_eq[i] = 0.0; // Prune dust
    final_sum += Q_eq[i];
}
for (double& q : Q_eq) q /= final_sum;

// Run one massive evaluation to prove the equilibrium
auto final_costs = sim.evaluate_crn_batch(Q_eq, Config::REPS_PER_ROUND *
Config::FINAL_EVAL_MULT, 999999);
double final_mix_cost = std::inner_product(Q_eq.begin(), Q_eq.end(), final_costs.begin(), 0.0);
double final_best_cost = *std::min_element(final_costs.begin(), final_costs.end());

std::cout << "\n=====\\n";
std::cout << " NASH EQUILIBRIUM SUPPORT P(h) | (Rebate a = 0) \\n";
std::cout << "=====\\n";
std::cout << std::left << std::setw(8) << "h" << std::setw(12) << "P(h)" << "Expected Cost\\n";
std::cout << std::string(40, '-') << "\\n";

for (int i = 0; i < num_strats; ++i) {
    if (Q_eq[i] > 0) {
        std::cout << std::left << std::setw(8) << strategies[i]
            << std::fixed << std::setprecision(4) << std::setw(12) << Q_eq[i]
            << std::fixed << std::setprecision(2) << final_costs[i] << "\\n";
    }
}

std::cout << "\\nEquilibrium Expected Cost : " << final_mix_cost << "\\n";
std::cout << "Best Response Pure Cost : " << final_best_cost << "\\n";
std::cout << "Exploitability Margin : " << (final_mix_cost - final_best_cost) << "\\n\\n";

return 0;

```

}